Submitted By:

Aneeb Ur Rehman – 2025-CYS-34

Assignment:

Pacman Game

Submitted To:

Sir Shahmeer Nawaz

For the fulfillment of,

Object-Oriented Programming Lab

Department of Computer Engineering

University of Engineering and Technology, Lahore

Pacman Game Project

Technical Report

Java Console Application

May 2026

1. Project Overview

This project is a console-based implementation of the classic Pacman arcade game written in Java. The player controls Pacman using keyboard input (W/A/S/D) and navigates a fixed 9x9 grid maze. The objective is to collect all food pellets ('.') on the board without being caught by the Ghost enemy.

The game ends in one of two ways:

- Win condition: Pacman collects every food pellet on the board.
- Loss condition: The Ghost occupies the same cell as Pacman.

The project is structured using object-oriented principles, with each game entity and the game loop separated into distinct classes.

2. Class Structure

The project consists of five classes, each with a focused responsibility:

2.1 Food

⬡ Food	
Represents a food pellet on the board. Stores its grid position.	
Fields	**Key Methods**
`int row` `int col`	`getRow()` `getCol()`

Note: While this class exists, food pellets are primarily tracked as '.' characters in the GameBoard's character array. The Food class is defined but not directly instantiated in the current game flow.

2.2 GameBoard

⬡ GameBoard	
Holds the 9x9 maze grid and manages the state of walls, food, and rendering.	
Fields	**Key Methods**

`char[][] structure`	`isWall(row, col)`
	`isFood(row, col)`
	`hasFood()`
	`eatFood(row, col)`
	`printBoard(p, g)`
	`getRows() / getCols()`

The maze is hard-coded as a 2D character array. '#' represents walls, '.' represents food, and ' ' is an empty/eaten cell. The printBoard() method overlays Pacman ('@') and Ghost ('G') on top of the structure.

2.3 Pacman

⬡ Pacman	
Represents the player character. Handles directional movement and score tracking.	
Fields	**Key Methods**

`int row`
`int col`
`int score`
	`move(direction, board)`
	`getRow()`
	`getCol()`
	`getScore()`

Movement is validated before being applied — Pacman cannot move outside bounds or into walls. When Pacman moves onto a food cell, the score increments by 1 and the cell is eaten.

2.4 Ghost

⬡ Ghost	
Represents the enemy. Moves randomly each turn using a random direction and jump distance.	
Fields	**Key Methods**

`int row`
`int col`
	`move(board)`
	`getRow()`
	`getCol()`

Each turn the Ghost picks a random direction (0–3) and a random jump of 1–5 cells. If the resulting position is in-bounds and not a wall, the Ghost teleports there. Otherwise, it stays in place. This makes the Ghost's behaviour erratic and non-deterministic.

2.5 PacmanGame

⬡ PacmanGame	
Entry point. Contains the main game loop, initializes all objects, and handles win/loss conditions.	
Fields	**Key Methods**

`PACMAN_START_ROW = 4`
`PACMAN_START_COL = 4`
`GHOST_START_ROW = 1`
`GHOST_START_COL = 1`
	`main(args)`

3. How the Game Works

3.1 Initialization

At startup, a GameBoard, a Pacman, and a Ghost are created with fixed starting positions. Pacman begins at the center of the maze (row 4, col 4) and the Ghost starts at the top-left interior cell (row 1, col 1).

3.2 Game Loop

The game runs in an infinite while loop. Each iteration performs the following steps in order:

- The current board state is printed to the console, showing Pacman ('@'), Ghost ('G'), walls ('#'), and food ('.').

- The player is prompted to enter a move: W (up), A (left), S (down), D (right). Input is case-insensitive.
- Pacman.move() is called — the new position is validated and food is consumed if present.
- Ghost.move() is called — the Ghost makes a random jump.
- Collision is checked: if Pacman and Ghost share the same cell, the game ends with a loss.
- Food check: if no '.' characters remain on the board, the game ends with a win.

3.3 Scoring

The player earns 1 point for each food pellet Pacman walks over. The final score is displayed at game over, regardless of outcome.

3.4 Board Layout

The 9x9 maze is symmetric and pre-defined. It features wall clusters that create corridors and pockets. The center cell (4,4) is intentionally empty (a space character) at the start, serving as Pacman's spawn point.

4. Known Issues

- Ghost teleportation: The Ghost can jump up to 5 cells in one move, passing through walls and food without interacting with them. This is not true pathfinding and can feel inconsistent.
- Food class is unused: The Food class is defined with a constructor and getters but is never instantiated anywhere in the game. Food is tracked solely via the char[][] array.
- No input validation: If the player types multiple characters or a non-WASD key, only the first character is read. Invalid keys are silently ignored (Pacman doesn't move).
- Single-step ghost check: Collision is only detected after both Pacman and Ghost have moved. If the Ghost jumps onto Pacman's previous position and Pacman moves away in the same turn, no collision is detected.

5. Future Improvements

5.1 Gameplay Enhancements

- Intelligent Ghost AI: Replace random jumping with BFS or A* pathfinding so the Ghost actively pursues Pacman through corridors.
- Multiple Ghosts: Add 2–4 ghosts with slightly different speeds or personalities (like Blinky, Pinky, Inky, Clyde in the original).

- Power Pellets: Add special pellets that temporarily allow Pacman to eat the Ghost, awarding bonus points.
- Levels & Difficulty Scaling: Increase Ghost speed or add more Ghosts as the player advances to new levels.
- Larger / Procedural Mazes: Generate mazes programmatically instead of hard-coding a single 9x9 board.

5.2 Technical Improvements

- GUI with Java Swing or JavaFX: Replace the text console with a graphical window for smoother gameplay and visual assets.
- Arrow key input: Use a library like JLine or lanterna to capture arrow keys in real-time without pressing Enter.
- Fix the Food class: Either remove it (since food is tracked in GameBoard) or refactor to use Food objects in a list for proper encapsulation.
- High Score Persistence: Save and load the high score to a file so it persists between sessions.
- Unit Testing: Add JUnit tests for Pacman.move(), Ghost.move(), and GameBoard boundary conditions.

6. Conclusion

This Pacman project successfully demonstrates core object-oriented programming concepts in Java — encapsulation, separation of concerns, and class interaction — within a simple but functional console game.

The five-class structure cleanly divides responsibilities: the board manages state, Pacman and Ghost are autonomous entities with their own movement logic, and PacmanGame orchestrates the loop. This design makes the codebase readable and extensible.

While the current version is limited to a fixed maze and a randomly-moving ghost, the architecture provides a solid foundation for the improvements listed above. Transitioning to a GUI, adding proper AI, and fixing the unused Food class would elevate this from a learning exercise to a polished mini-game.

Overall, the project is a well-structured starting point that demonstrates good object-oriented instincts and an understanding of game loop design.

PACMAN GAME – ARCHITECTURE DIAGRAM